Assignment 1: Train a minimal language model from scratch

Building Large Foundation Models from Scratch · WS 2026/27

Out 6 Oct · Due Wed 21 Oct 2026, 23:59

Asst. Prof. Dr. Martin Pawelczyk · martin.pawelczyk@univie.ac.at · Moodle

The brief summary

You will build every part of a small language model yourself: a byte-level BPE tokenizer, a Transformer with the 2024 block (RMSNorm, RoPE, SwiGLU, grouped-query attention), cross-entropy, AdamW, a cosine schedule, a training loop with checkpointing, and a sampler. You will then train the reference model (22.7M parameters, $2 \cdot 10^8$ tokens of TinyStories) on a free Colab GPU, measure how fast it runs against the roofline, sweep the learning rate, run one ablation you predicted in advance, and write a three-page report. Tests are provided for every function. No frameworks: `torch` tensors and `torch.autograd` only.

1 Logistics

- **Compute**: Google Colab (the free T4 is enough; Pro is faster), or any GPU you have. The whole assignment needs about 2–3 GPU-hours on a free T4. Budgets are stated in FLOPs.
- **Starter kit** on Moodle and at https://github.com/MartinPawelczyk/BLfS/tree/main: the test suite with `adapters.py`, a Colab notebook, a data script, and the report template (layout in Section 2).
- **Data**: TinyStories [Eldan & Li, 2023] as `tinystories_train.txt` (2.1 GB) and `tinystories_valid.txt` (22 MB); a 100 MB sample of OpenWebText for the tokenizer comparison; and, from Lab 1 on, `train.bin`/`val.bin`: the training text tokenised with the course tokenizer ($V = 10{,}000$, about 540M tokens, `uint16`).
- **Deadline** Wed 26 Oct, 23:59, on Moodle.
- **Collaboration and AI assistants**: discuss freely, but write your own code. Do not read other people's BPE or Transformer code, and do not use AI assistants to generate your solution; they are fine for conceptual and API questions. Cite anything you took from elsewhere. The mid-term exam contains questions that are easy if you did this yourself.

2 Setup

2.1 Repository layout

```
a1/
  lm/
  tests/                    provided, do not modify:
    adapters.py             you fill in the bodies so the tests call your code
    test_tokenizer.py  test_model.py  test_nn_utils.py
    test_optimizer.py  test_data.py
  notebooks/setup.ipynb     Colab: mount Drive, fetch data, run tests, start a run
  scripts/prepare_data.py   tokenise the corpus into uint16 memmaps
  results.json              written by your training script (Section 6)
  report/report.tex         the template: three pages plus figures
```

2.2 Environment

Python $\geq$ 3.11, `uv` for the environment, PyTorch 2.x with `numpy`, `regex`, `einops`, `jaxtyping`, `pytest`. On Colab everything except `einops`/`jaxtyping` is preinstalled; `setup.ipynb` installs the rest. Run the tests locally on CPU (they take under a minute); use the GPU only for training.

```
uv sync                     # or: pip install -e .
uv run pytest -q            # all tests; -k tokenizer, -k model, ... for subsets
```

2.3 Colab workflow

Free Colab gives a T4 (16 GB, fp16 tensor cores, *no bf16*) for a few hours at a time; sessions disconnect. The rules that follow from that:

- Mount Drive; keep `train.bin`, `val.bin`, checkpoints and `results.json` there. Copy `train.bin` to the local disk at the start of a session (it is faster to read from).
- Checkpoint every few minutes and resume from the last checkpoint on restart. Make this work *before* the first long run: kill the runtime mid-run, restart, continue.
- Mixed precision: fp16 autocast with a `GradScaler` on the T4 (it has no bf16), bf16 autocast on an L4 or A100; the loss in fp32.

```
with torch.autocast(device_type="cuda", dtype=torch.float16):   # bf16 on L4/A100
    logits = model(x)
loss = cross_entropy(logits.float(), y)
scaler.scale(loss).backward()                                    # GradScaler, fp16 only
```

- Log the GPU type and tokens/s in every run; both go into `results.json`.

3 Part 1: Byte-pair-encoding tokenizer

3.1 Specification

Implement a byte-level BPE tokenizer with GPT-2's pre-tokenisation (see Lecture 1!).

Training. `train_bpe(input_path, vocab_size, special_tokens) -> (vocab, merges)` with `vocab: dict[int, bytes]` and `merges: list[tuple[bytes, bytes]]`, where

1. the initial vocabulary is the 256 bytes plus the special tokens – in that order;
2. the text is split on special tokens first, then pre-tokenised with the GPT-2 regular expression

   ```
   PAT = (r"""'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+"""
          r"""| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+""")
   ```

   (use the `regex` package); merges never cross pre-token boundaries and never involve special tokens;
3. at each step the most frequent adjacent pair is merged; ties are broken by the lexicographically greater pair (compare the two `bytes` objects as a tuple); the new token is appended to the vocabulary and the pair to `merges`;
4. training stops when the vocabulary reaches `vocab_size`.

Encoding and decoding. A class `Tokenizer(vocab, merges, special_tokens=None)` with

- `from_files(vocab_path, merges_path, special_tokens)`, a class method;
- `encode(text) -> list[int]`;
- `encode_iterable(iterable) -> Iterator[int]`, which streams a file without loading it;
- `decode(ids) -> str`; invalid UTF-8 decodes with `errors="replace"`.

Encoding applies the merges in training order (by rank) inside each pre-token; special tokens are matched before pre-tokenisation, longest match first.

3.2 Performance

A naive implementation (see Lecture 1) recounts all pairs after every merge and takes hours on TinyStories. Optimise this naive implementation.

Part 1 deliverables

1. `tests/test_tokenizer.py` green: `test_train_bpe`, `test_train_bpe_special_tokens`, `test_train_bpe_merge_order`, `test_roundtrip_unicode`, `test_special_tokens`, `test_encode_merge_order`, `test_encode_iterable_memory`.
2. A tokenizer trained on TinyStories with $V = 10{,}000$ and special token `<|endoftext|>`. Report: training time and peak memory, the longest token, the first twenty merges, and bytes per token on the TinyStories validation text and on the OpenWebText sample (Lecture 1's "number that matters"). If training on the full 2.1 GB takes more than 30 minutes, train on the first 200 MB.
3. Encoding throughput in MB/s, and the time to tokenise the validation set.

4 Part 2: The model and the training utilities

All modules subclass `torch.nn.Module`, hold parameters as `nn.Parameter`, and use no `torch.nn` layers except `nn.Module` and `nn.Parameter`. Store linear weights as $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and apply them as `einsum(x, W, "... i, o i -> ... o")`; the tests assume this convention.

4.1 Modules

`Linear(in_features, out_features)`
no bias; init $\mathcal{N}(0, \sigma^2)$ with $\sigma^2 = 2/(d_{\text{in}}+d_{\text{out}})$ truncated at $\pm 3\sigma$ (`torch.nn.init.trunc_normal_`).

`Embedding(num_embeddings, embedding_dim)`
lookup by integer ids; init $\mathcal{N}(0, 1)$ truncated at ± 3.

`RMSNorm(d_model, eps=1e-5)`
$y = x/\sqrt{\frac{1}{d}\sum_i x_i^2 + \epsilon} \odot g$, gain g initialised to 1; statistics computed in fp32, result cast back to the input dtype.

`SwiGLU(d_model, d_ff)`
$W_2\,(\text{SiLU}(W_1 x) \odot W_3 x)$ with $\text{SiLU}(z) = z\sigma(z)$; $d_{\text{ff}} = \frac{8}{3} d_{\text{model}}$ rounded to the nearest multiple of 64 (1344 for $d_{\text{model}} = 512$).

`RotaryPositionalEmbedding(theta, d_k, max_seq_len)`

rotates pairs (x_{2i}, x_{2i+1}) at position m by $m\theta_i$, $\theta_i = \Theta^{-2i/d_k}$, $\Theta = 10{,}000$; precompute cos/sin tables as buffers; `forward(x, token_positions)`. Applied to queries and keys after the head split and not to values.

`softmax(x, dim)`
numerically stable (subtract the maximum).

`scaled_dot_product_attention(q, k, v, mask=None)`
$\text{softmax}(qk^\top/\sqrt{d_k} + M)\,v$ with a boolean mask (`True` = may attend), masked positions set to $-\infty$ before the softmax, softmax in fp32.

`MultiheadSelfAttention(d_model, num_heads, num_kv_heads, rope)`
causal by default; `num_kv_heads=None` and `rope=None` by default; n_q query heads share n_{kv} key/value heads (grouped-query attention, Lecture 3); $n_{kv} = n_q$ when not given; output projection W_O.

`TransformerBlock(d_model, num_heads, d_ff, ...)`
pre-norm: $h = x + \text{Attn}(\text{RMSNorm}(x))$, $y = h + \text{SwiGLU}(\text{RMSNorm}(h))$.

`TransformerLM(vocab_size, context_length, d_model, num_layers, num_heads, d_ff)`
embedding, n_{layers} blocks, final RMSNorm, untied output head; `forward(ids) -> logits` of shape `(batch, seq, vocab)` in fp32.

`count_params(model)` and `count_flops(config, seq_len)`
the accounting from Lecture 2; `count_flops` returns forward FLOPs per token, including the head and the attention scores, and training FLOPs per token (3×).

4.2 Training utilities

`cross_entropy(logits, targets)`
mean negative log-likelihood over all positions; subtract the maximum logit before exponentiating (log-sum-exp); inputs in fp32.

`AdamW(params, lr, betas=(0.9, 0.95), eps=1e-8, weight_decay=0.1)`
a `torch.optim.Optimizer` subclass implementing the update from Lecture 2 with decoupled weight decay: $m \leftarrow \beta_1 m + (1-\beta_1)g$, $v \leftarrow \beta_2 v + (1-\beta_2)g^2$, bias-corrected $\hat{m}, \hat{v}$, $\theta \leftarrow \theta - \alpha(\hat{m}/(\sqrt{\hat{v}} + \epsilon) + \lambda\theta)$. Verify your implementation against `torch.optim.AdamW` on a toy problem.

`lr_cosine(step, lr_max, lr_min, warmup_steps, total_steps)`
linear warmup to $\alpha_{\max}$, cosine decay to $\alpha_{\min}$ at `total_steps`, constant afterwards.

`clip_grad_norm_(params, max_norm, eps=1e-6)`
rescale all gradients so that their global ℓ_2 norm is at most `max_norm`; return the norm before clipping (log it).

`get_batch(data, batch_size, context_length, device)`
`data` is a `np.memmap` of `uint16` token ids; sample `batch_size` random offsets, return inputs `ids[i:i+L]` and targets `ids[i+1:i+L+1]` as `int64` tensors on `device`.

`save_checkpoint(model, optimizer, step, path)`, `load_checkpoint(path, ...)`
the latter takes the model and optimizer too and returns the step; together they hold everything needed to resume with an identical schedule; use `torch.save` on the state dicts.

`generate(model, tokenizer, prompt, max_new_tokens, temperature, top_p)`
sample one token at a time with temperature and nucleus (top-p) sampling; stop at `<|endoftext|>`.

Part 2 deliverables

All tests in `test_model.py`, `test_nn_utils.py`, `test_optimizer.py` and `test_data.py` green, and `count_params` approximately reproducing Table 1.

5 Part 3: Training the reference model

5.1 Before any long run

1. Overfit one batch: loss → below 0.1 within a few hundred steps. If not, the model or the optimizer is broken.
2. The gradient norm is finite and roughly stable.
3. One profiled step (`torch.profiler`, Lecture 4): note what takes the time.

Setting	Value
vocabulary V	10,000
context length L	256
d_{model}	512
n_{layers}	4
heads n_q/n_{kv}	16 / 16 (head dim 32)
d_{ff}	1344
RoPE Θ	10,000
embeddings	untied
parameters	22,696,448: blocks 12,455,936, embedding 5,120,000, head 5,120,000
forward FLOPs per token	37,240,832 (head 27 %)
training FLOPs per token	$1.117 \cdot 10^8$
tokens D, steps, batch	$2 \cdot 10^8$, 24,414, $32 \times 256 = 8{,}192$ tokens
training FLOPs	$2.2 \cdot 10^{16}$

Table 1: **The reference configuration (Lecture 2).** Your `count_params` and `count_flops` should roughly reproduce these numbers.

4. Kill the runtime during a short run and resume from the checkpoint. The loss curve must continue where it stopped.

5.2 The runs

Fixed for all runs: the configuration in Table 1, AdamW with $\beta = (0.9, 0.95)$, weight decay 0.1, gradient clipping at 1.0, warmup 5 % of the steps, cosine decay to $0.1\,\alpha_{\max}$, batch 32×256, seed 0, validation loss on the first 2^{20} tokens of `val.bin`, computed in fp32.

1. **Learning-rate sweep** at one fifth of the budget (4,883 steps, about 10 minutes each on a T4): $\alpha_{\max} \in \{3 \cdot 10^{-4}, 10^{-3}, 3 \cdot 10^{-3}, 10^{-2}\}$. Plot final validation loss against $\alpha_{\max}$ on a log axis. Expect a U and possibly a divergence at the top; keep the loss curves of all four.

2. **Reference run** at the best rate for the full 24,414 steps ($2 \cdot 10^8$ tokens, $2.2 \cdot 10^{16}$ FLOPs). On a T4 this takes 30–60 minutes. the exact target is announced on Moodle after the reference solution has been run on a T4. Save the final checkpoint; it is part of the submission.

3. **Efficiency table**: tokens/s of the reference run's step, measured over 200 steps after warm-up with `torch.cuda.synchronize()`, for the eager fp32 baseline and after each change in this order: autocast (fp16 on T4, bf16 otherwise), `F.scaled_dot_product_attention` in place of your `sdpa`, `torch.optim.AdamW(fused=True)` in place of yours, `torch.compile`. Convert each to MFU with `count_flops` and the peak of your GPU (T4: 65 TFLOP/s fp16; L4: 121 TFLOP/s bf16; A100: 312 TFLOP/s bf16). Your own `sdpa` and `AdamW` must pass the tests; the training runs may use the library versions.

4. **One ablation**, at the sweep budget (4,883 steps) and the best learning rate, against the matching sweep run. Choose one: tie the embedding and the head; replace SwiGLU by a ReLU MLP with $d_{\text{ff}} = 4d_{\text{model}}$ (equal parameters); remove RMSNorm; remove warmup; grouped-query attention with $n_{kv} = 4$. Write down your predicted change in validation loss *before* running, and put the prediction next to the measurement.

5. **Samples**: five generations of up to 256 tokens from the reference checkpoint, prompts of your choice, at temperature 0.7 and $p = 0.9$, plus one at temperature 1.5 to see what happens.

Budget

Sweep 4×10 min, reference run 30–60 min, ablation 10 min, efficiency table 10 min: about two T4-hours in total. Do the sweep and the ablation first; the reference run should be a one-hour Colab session.

6 Report

Three pages of text using `report/report.tex`. Please follow this structure in this order:

R1 **Tokenizer**: the numbers from Part 1, the twenty merges, and two sentences on what the bytes-per-token difference between TinyStories and OpenWebText tells you.

R2 **Sanity checks**: initial loss, the overfit-one-batch curve, the resume test.

R3 **Learning-rate sweep**: the plot and the chosen rate.

R4 **Reference run**: train and validation loss curves (log-scale x), final validation loss, GPU type, tokens/s, MFU.

R5 **Efficiency table** with the five rows above; one paragraph on which change moved which bar of your profile (Lecture 4's checklist).

R6 **Ablation**: prediction, measurement, and an explanation of the difference.

R7 **Samples** and two sentences on their quality.

R8 **What went wrong**: the bug that cost you the most time, and how you found it. This is to improve the lecture for the coming years.

7 Submission and grading

Upload one archive to Moodle: the repository (without data), `results.json`, the report PDF, and the notebook you trained with.

```
{ "tokenizer": {"vocab_size": 10000, "train_seconds": 412,
                "peak_memory_gb": 3.1, "encode_mb_per_s": 2.4,
                "bytes_per_token": {"tinystories_val": 4.12, "owt_sample": 3.35},
                "longest_token": "..."},
  "model": {"params": 22696448, "flops_per_token_fwd": 37240832},
  "gpu": "Tesla T4", "precision": "fp16",
  "lr_sweep": [{"lr": 3e-4, "steps": 4883, "val_loss": 1.83}, ...],
  "reference_run": {"lr": 1e-3, "steps": 24414, "tokens": 2e8,
                    "flops": 2.2e16, "val_loss": 1.43,
                    "tokens_per_s": 21800, "mfu": 0.21, "wallclock_s": 3130},
  "efficiency": [{"config": "eager fp32", "tokens_per_s": 6100, "mfu": 0.06},
                 ...],
  "ablation": {"name": "tied_embeddings",
               "predicted_delta": 0.03, "measured_delta": 0.018},
  "samples": ["...", "..."] }
```

8 Known pitfalls

- *My loss is `nan` after a few steps on the T4.* fp16 overflow: use the `GradScaler`, keep the loss and the softmax in fp32, clip gradients. Or train in fp32 at half the speed.
- *Attention returns `nan` for some rows.* A row whose mask is entirely `False` has a softmax over $-\infty$ only. With a causal mask this cannot happen; check your mask orientation (`True` = attend).
- *Training is slow (below* 10,000 *tokens/s on a T4).* Profile before guessing. Typical culprits: a Python loop over heads, `.item()` every step, `get_batch` building tensors on the CPU one row

at a time, the loss computed in float64.

- *The tokenizer takes hours.* You are recounting pairs from scratch after every merge, or pre-tokenising with a regex over the whole 2 GB string. Count pre-tokens first; chunk and parallelise.
- *Colab disconnected at step* 20,000. That is what checkpoints are for. If you lost the run, you skipped step 5 of Section 5.1.
- *Which lectures do we need?* L1 for Part 1; L2 and L3 for Part 2; L2 and L4 for Part 3. The slides contain hints and the code skeletons.

References: Sennrich et al. (2016), Neural machine translation of rare words with subword units; Radford et al. (2019), Language models are unsupervised multitask learners (the GPT-2 tokenizer); Eldan & Li (2023), TinyStories; Vaswani et al. (2017); Su et al. (2021), RoFormer; Shazeer (2020), GLU variants; Loshchilov & Hutter (2019), AdamW; Ainslie et al. (2023), GQA.